

○ TESTING INTELLIGENCE

[API](#)
[FAQ](#)
[SƠ ĐỒ](#)
[HOẠT ĐỘNG TI](#)
[TẠO KEY](#)
[API THAM CHIẾU](#) · [V1 TƯƠNG THÍCH](#) · [V2 ĐÃ TRIỂN KHAI, ĐANG NGHIỆM THU](#)

Tích hợp TI vào CI. Độc đúng bằng chứng.

Hai hợp đồng API riêng, cùng một dịch vụ, cùng một kho job: **v1** (đường /v1/... — trạng thái LIVE · FROZEN trên <https://d1tibdarzwmw3jq.cloudfront.net>) và **v2** (đã triển khai theo receipt 2026-09-14, path /v2/artifact-jobs ; chưa nghiệm thu end-to-end). Gọi phiên bản nào cũng phải chạy được; job tạo qua một mặt đọc được qua mặt kia. Bốn bước CI bên dưới dùng v2 cho job mới; hợp đồng v1 vẫn tương thích ở mục riêng. Hai hợp đồng evaluations / answer-change legacy ghi riêng ở nửa dưới.

Tài liệu chi tiết hợp đồng API: Wiki của repo (cần quyền truy cập repo TechX-Corp/testing-intelligence) — <https://github.com/TechX-Corp/testing-intelligence/wiki>. Trang này vẫn *tự đủ* về ranh giới phiên bản: hai mục dưới đây là nguồn đúng khi bạn chỉ đọc portal.

00 · PHIÊN BẢN HỢP ĐỒNG

Hai trục số · bốn nhãn trạng thái · cả hai sống cùng lúc.

Hai trục không trộn: (1) **hợp đồng API** v1 / v2 trên path — chỉ tăng khi phá hợp đồng đang phục vụ; (2) **bản phát hành** TI <YYYY.MM.DD>-<sha7> mỗi lần publish (release notes / trang docs). Không phải số Runtime AgentCore (TIJobRunner vN do AWS đánh) và không phải schema_version trong body. Nhãn trạng thái đúng bốn giá trị: LIVE · FROZEN · IN DEVELOPMENT · PROPOSED . Không công bố ngày Sunset — lịch tắt công bố riêng sau này. Không tuyên bố quá wiki: chưa nghiệm thu end-to-end; policy LOG_ONLY chỉ là đã quan sát; bucket bằng chứng chưa immutable; có bản ghi publication ≠ reuse đã chạy.

Receipt triển khai 2026-09-14: source fbdd8dfc6632b0994ac360bc4871b4ebc26ae673 , image build efabf57dcd8c , AgentCore Runtime 18 . Đây là receipt triển khai, chưa nghiệm thu end-to-end: authenticated jobs, hai tenant, receiver ACK và K1/K2 cần bằng chứng riêng. [Theo dõi nghiệm thu #97](#).

v1

/v1/...

LIVE · FROZEN — web hiện tại, vẫn hỗ trợ, không đổi hành vi

v1 là hợp đồng đang phục vụ thật trên <https://d1tibdarzwmw3jq.cloudfront.net> — đúng thứ bên gọi đang dùng hôm nay, vì thế trạng thái là LIVE và FROZEN . Mọi đường /v1/testing/... hiện có giữ nguyên. Trạng thái job trên v1 chỉ gồm queued · running · completed · failed . Một job timed_out hoặc cancelled (trong kho) **chiếu thành state: failed** khi đọc qua v1. **GET job durable đã FAILED trả 503** kèm error — không phải 200. Payload v1 không mang trace_id , result_status , lifecycle , attempt , worker_lease , heartbeat_at , started_at , completed_at hay retryable .

POST artifact qua /v1/testing/changes vẫn nhận 202 và mang header Deprecation: true cùng Link: </v2/artifact-jobs>; rel="successor-version" , cùng preferred_poll trở sang v2. **Không có header Sunset / không có ngày tắt.**

v2

/v2/artifact-jobs

Đã triển khai · chưa nghiệm thu end-to-end

v2 đã triển khai theo receipt 2026-09-14; triển khai không đồng nghĩa nghiệm thu. post /v2/artifact-jobs và GET /v2/artifact-jobs/{job_id} . Cùng kho job với v1: job tạo qua v2 đọc được qua v1 (chiếu xuống hợp đồng v1) và ngược lại. **Job FAILED vẫn đọc được trả 200 kèm state và error** (5xx chỉ khi TI không phục vụ được). Trạng thái gồm đủ sáu giá trị: queued · running · completed · failed · timed_out · cancelled — partial **không** là state; đó là result_status PARTIAL trong tập đóng PASS · FAIL · PARTIAL · INCONCLUSIVE . timed_out / cancelled chỉ tồn tại trên v2 (không chiếu thành failed). trace_id do **server cấp lúc nhận** và ghi đè giá trị caller gửi kèm.

Tóm tắt khác biệt: FAILED đọc được = **503 trên v1, 200 + state trên v2.**

KHÁM PHÁ HỢP ĐỒNG VÀ THAO TÁC NÂNG CAO

Các đường sau cần cùng bearer credential như job; chỉ đọc/ghi trong phạm vi danh tính và repository được cấp. Dùng Authorization header trong API client, không truyền key qua URL. Vai reviewer không tự có quyền chạy hay đọc job.

- GET /v2/evaluation-capabilities : check schema, capabilities và cách chọn artifacts[].capability_checks ; availability không chứng minh runtime đạt.
- GET /v2/release-contracts : request schemas, evaluator allowlist và worker availability; GET /v2/providers : provider đã cấu hình trong phạm vi của bạn.
- POST /v2/campaigns và GET /v2/campaigns : lưu/đọc định nghĩa và coverage quan sát; không khởi chạy job mới.
- POST /v2/artifact-jobs/{job_id}/trace-evaluations : chọn invocation đã ghi của job, trả 202 operation ID; trace phải có bằng chứng AWS thực.
- POST /v2/artifact-jobs/{job_id}/evidence-deliveries : tạo outbox từ evidence snapshot, trả 202; transport delivery không phải receiver ACK.
- GET /v2/operations?job_id=... và GET /v2/operations/{id} : đọc trạng thái, scores và lịch sử; POST /v2/operations/{id}/actions : replay hoặc yêu cầu withdrawal theo schema.

403: thiếu quyền; 404: không tồn tại hoặc ngoài phạm vi; 422: contract/state không hợp lệ; 503: cấu hình/provider không sẵn sàng. Lỗi async nằm trên operation. [Hợp đồng chi tiết theo source triển khai \(cần quyền repo\)](#) · [OpenAPI trong repo · nhánh main \(cần quyền repo; có thể khác receipt triển khai\)](#) · [Monitor job hiện tại](#).

▼ Mục lục API

PHIÊN BẢN

[00 v1 tương thích · v2 đã triển khai](#)

[API v1](#)

[API v2](#)

BẮT ĐẦU

[01 Bắt đầu nhanh](#)

CREDENTIAL

[02 Lấy key](#)[03 Xác thực](#)

GỬI ARTIFACT

[04 Artifact của PR](#)[Review & Memory \(reviewer\)](#)

ĐỌC KẾT QUẢ

[05 Đọc kết quả](#)[06 Khi bị từ chối](#)

NÂNG CAO / LEGACY

[07 Legacy · ghi](#)[08 Legacy · đọc](#)[09 Trạng thái](#)

ĐƯỜNG ĐI

[12 Đường đi](#)[Chín nhóm năng lực](#)[Vai AgentCore](#)[Bản đồ điểm gắn](#)

01 · BẮT ĐẦU NHANH

Bốn bước, chép là chạy: credential → bấm → POST → poll.

Khởi lệnh dưới đây dùng [hợp đồng v2](#) (/v2/artifact-jobs): terminal đọc được trả 200 , phải kiểm state và kết quả. V1 LIVE · FROZEN vẫn hỗ trợ ở [mục tương thích](#), failed terminal trả 503 . Mỗi bước một khối lệnh có nút chép, dùng hai biến TI_URL và TI_KEY , đúng tên biến mà client scripts/verify-artifact-api.py đọc.

- 01** Kết nối: đặt địa chỉ và credential. Key cá nhân lấy ở mục [Lấy key](#) (đăng nhập Microsoft, chỉ hiện một lần). Với key này, đội bạn tự viết CI và gọi API ngay: không cần đăng ký repository với TI, không cần workflow do TI quản. Chỉ khi muốn bỏ secret tĩnh (GitHub OIDC) hoặc gắn

canonical binding mới cần cấu hình phía TI: xem [Xác thực](#). Lệnh cuối chỉ kiểm credential có được nhận không, không ghi gì.

- Bước 1 · đặt TI_URL, TI_KEY và kiểm credential

```
export TI_URL=https://d1tibdarzwmw3jq.cloudfront.net # địa chỉ TI đang
chạy; trong CI đặt qua vars.TI_URL
export TI_KEY=ti_... # key cá nhân, không
dán vào mã nguồn hay log

# 200 = credential được nhận, thân là danh sách job của chính bạn (có thể
rỗng); 422 AUTHORITY_INSUFFICIENT = thiếu hoặc sai credential
curl -sS -o /dev/null -w '%{http_code}\n' "$TI_URL/v2/artifact-jobs?limit=1"
-H "authorization: Bearer $TI_KEY"
```

Chép

02 Băm nội dung. TI đối chiếu sha256 với đúng byte UTF-8 của content ; băm lệch là 400 UNDECLARED_FIELD_REJECTED (thân chỉ nêu tên trường trong rejected_fields[] , không nêu giá trị) trước khi có bất kỳ đánh giá nào. version là số có dấu chấm (1.0); đổi nội dung thì tăng version và tính lại băm.

+ Bước 2 · băm SHA-256 của artifact

03 Gửi artifact, nhận 202. Body đầy đủ (schema, giới hạn, baseline , canonical_link , idempotency_key) ở mục [Artifact của PR](#). Khối dưới ghi body ví dụ ra request.json rồi POST đúng một lần, giữ phản hồi trong submit.json ; thay body bằng artifact của bạn với băm ở bước 2. Phản hồi 202 mang job_id , state và đường poll ; header x-correlation-id cũng là job_id. Mỗi lần POST là một job mới, nên bước 4 không gửi lại.

+ Bước 3 · POST artifact đúng một lần → submit.json

04 Poll rồi đọc (hợp đồng v2). Gọi đúng đường poll cùng origin tới khi hết 202. 200 có thể là completed , failed , timed_out hoặc cancelled ; phải đọc state . 5xx là service lỗi; 404 che job không thuộc danh tính. Khối dưới kiểm job_id 32 hex và đường poll v2, timeout 15 giây mỗi lượt, deadline 300 giây (TI_POLL_DEADLINE). Mọi kết thúc không completed hoặc thiếu result đều thoát khác 0; result.json giữ nguyên. Completed không phải PASS; đọc result_status , từng check, khoảng trống bằng chứng và khuyến nghị trước khi quyết định.

+ Bước 4 · poll an toàn tới trạng thái terminal → result.json

Tùy chọn, không bắt buộc: repo TI có client tương thích v1 FROZEN scripts/verify-artifact-api.py submit/poll và ghi báo cáo (failed trả 503). Tập đó không có sẵn trong CI của đội bạn; bốn khối lệnh ở trên là đủ để tích hợp, client chỉ là đường thay thế nếu đội bạn chọn mang nó theo.

+ Tùy chọn · client tham chiếu của TI

Đọc result.json theo bốn lớp ở mục [Đọc kết quả](#): trạng thái job, check đã chạy, candidate máy sinh còn chờ, khuyến nghị. completed là job đã kết thúc, không phải artifact được duyệt.

02 · LẤY KEY

Một key cho một người, dùng được 365 ngày.

Đăng nhập bằng tài khoản Microsoft của TechX, chép key về, xong. Key chỉ hiện một lần. Mất key thì đăng nhập lại để lấy key mới.

Đăng nhập Microsoft để tạo key

- 01** Đăng nhập. Bấm nút ở trên. Trang hiện một mã ngắn và một đường dẫn của Microsoft. Mở đường dẫn, nhập mã, đăng nhập bằng tài khoản TechX. Trang này tự nhận kết quả và hiện key.
- 02** Cắt key. Chép key vào nơi giữ bí mật của bạn, ví dụ biến môi trường TI_KEY trên máy hoặc secret của CI. Không dán key vào chat hay vào mã nguồn.
- 03** Gọi API. Gửi key trong header Authorization: Bearer <key> ở mọi lượt gọi. Key mang vai evaluator trên tenant đang mở. Ba khối lệnh ở mục kế tiếp dùng đúng key này.

Key hết hạn sau 365 ngày kể từ lúc cấp; ngày hết hạn hiện ngay cạnh key lúc cấp. Gần hết hạn thì đăng nhập lại, lấy key mới và thay vào chỗ cũ; key cũ tự hết hiệu lực khi đến hạn. Lỡ để lộ key thì lấy key mới ngay và báo người vận hành TI thu hồi key cũ, kèm phần giữa của key (mã nhận dạng, không phải bí mật). Ứng dụng chạy tự động không cần bấm nút: xem mục [Xác thực](#).

03 · XÁC THỰC

Một header, ba loại danh tính đã xác minh.

Mọi lượt gọi mang header `Authorization: Bearer ...`. Vai và phạm vi được tra ở phía server; bên gọi không tự khai được.

GIÁ TRỊ	HỢP VỚI	LẤY Ở ĐÂU
access token Entra ID	luồng tự động, hệ thống chạy lâu, agent	trong terminal: <code>uv run python -m transports.keys.login</code> (device code, in token ra stdout); hoặc MSAL với cùng client id và scope; hoặc <code>az account get-access-token --scope api://<app-id-của-TI>/access_as_user</code> nếu tài khoản đã được cấp quyền
key <code>ti_<id>_<secret></code>	script cần một bí mật tĩnh, thử tay	trang key (đăng nhập Microsoft, dùng 365 ngày); hoặc <code>POST /keys</code> với token Entra; hoặc trên máy có mã nguồn TI: <code>uv run python -m transports.keys.login --key</code>
GitHub Actions OIDC token	CI submit/poll artifact của đúng repository đã được TI duyệt	workflow đặt <code>permissions: id-token: write</code> , rồi chạy client <code>scripts/verify-artifact-api.py --auth github-oidc</code> . Client mint token ngắn hạn cho từng lời gọi; không cần giữ một TI key dùng chung

Token Entra phải qua chữ ký, issuer, audience, tenant và scope. GitHub OIDC bị ràng buộc fail-closed vào repository ID/owner ID, repository, environment, ref, workflow và SHA đã được TI cấu hình; branch hay workflow tùy ý không được uỷ quyền. Với reusable workflow, workflow được gọi phải cùng repository và pin bằng full commit SHA. CI chỉ đóng gói, submit và poll kết quả; nó không chọn policy đánh giá hay tạo ra kết quả pass.

Token Entra và key do trang này cấp là hai cơ chế xác thực cho cùng subject `entra:<oid>`, không phải hai danh tính người dùng. GitHub CI là một principal repository-bound riêng; `on_behalf_of` chỉ ghi authority đã cấu hình và không cấp quyền đọc như một người dùng. Nếu body tự thêm `role`, service từ chối thay vì tin trường đó.

TUỶ CHỌN NÂNG CAO · GITHUB ACTIONS OIDC VỚI PROTECTED ENVIRONMENT ĐÃ ĐƯỢC TI DUYỆT

Không bắt buộc. Đường thường của CI là key cá nhân hoặc token Entra đặt trong secret của CI, gọi thẳng hai endpoint ở mục Bắt đầu nhanh mà không cần TI duyệt gì thêm. OIDC dành cho đội muốn bỏ hẳn secret tĩnh và chấp nhận một lần TI duyệt binding repository.

+ [Tuỳ chọn nâng cao · reusable workflow GitHub OIDC \(YAML\)](#)

Lưu recipe này thành reusable workflow trong repository dev. Caller phải gọi file đó bằng `owner/repo/.github/workflows/ti-called.yml@FULL_COMMIT_SHA` cùng repository, truyền input `submission_json` là toàn bộ ArtifactSubmission JSON từ bước đóng gói không đặc quyền (schema bên dưới), và cho phép `id-token: write`. Source phải ghi đúng repository/PR/base/head thực tế; không nội suy JSON vào shell hoặc chạy code từ PR trong job có OIDC. Bước materialize chỉ ghi dữ liệu; client kiểm tra payload/hash rồi submit và poll.

Setup một lần: vendor bản `scripts/verify-artifact-api.py` đã được TI duyệt vào repository dev, review/merge cùng caller workflow trên default branch và bảo vệ các file này bằng quy trình review. Checkout ở trên chỉ đọc chính repository dev tại `github.workflow_sha` (commit bất biến của caller workflow), không phải PR head; không cần quyền đọc private repository TI hay PAT. Caller ngoài default branch bị bỏ qua. Client update phải qua review trước khi dùng.

`TI_URL` và `TI_OIDC_AUDIENCE` là biến không bí mật của protected environment. Trước khi chạy, TI phải duyệt binding của repository ID/owner ID, caller workflow/ref/event, environment, audience và full SHA của reusable workflow; việc copy recipe không tự cấp quyền. Phần SHA trong `uses` của caller phải là full commit chứa reusable workflow đã duyệt, không phải branch/tag. Recipe này cần hoàn tất setup và binding trước khi chạy; nó không phải bằng chứng nghiệm thu CI live.

04 · ARTIFACT CỦA PR

Một đối tượng đổi version. Hệ thống trả chín câu.

Mặt Testing Intelligence theo mô hình năng lực (Confluence *Testing Intelligence in Engineering Factory*, mục "32. Definition of Done"): gửi cái gì đổi, nhận lại cái gì bị ảnh hưởng, cần test nào, đã chạy gì, hỏng gì, có bằng chứng gì, cổng nào bị chặn, ai quyết, và làm gì tiếp. Cùng key, cùng header `Authorization`, cùng bao thư lỗi. Test có tài liệu (nội dung artifact, hợp đồng scenario) được Agent Evaluation Runner trên AWS Bedrock AgentCore đo; test không có tài liệu và không có

quan sát là `not_evaluable` , không bao giờ là `pass` . **Job artifact**

mới: [POST /v2/artifact-jobs](#) . Đường `/v1/testing/changes` bên dưới là hợp đồng v1 FROZEN (failed = 503; không có `timed_out/cancelled` trên wire).

POST `/v2/artifact-jobs`

v2 · đã triển khai, đang nghiệm thu — nhận `ArtifactSubmission`, trả 202 + poll v2

Cùng schema body artifact như mục kế (schema_version, source, artifacts, policy). Phản hồi 202 mang `job_id` , `state` , `poll = /v2/artifact-jobs/{job_id}` , `trace_id` . Poll v2: 202 khi `queued/running`; **200 khi terminal** (kể cả `state: failed · timed_out · cancelled`) kèm `error / result` và metadata đầy đủ. 5xx chỉ khi TI không phục vụ được.

EVIDENCE, COVERAGE VÀ DỮ LIỆU LEARNING

`GET /v2/testing-runners` liệt kê 8 loại runner và profile thực sự có cấu hình trong phạm vi. Chọn profile qua `capability_checks[].probe_ref` ; mỗi receipt giữ loại executor, phiên bản, digest và scope.

`GET /v2/artifact-jobs/{job_id}/evidence-evaluation` trả integrity, binding và các operation eval của đúng job. Digest hợp lệ chưa chứng minh câu trả lời đúng. Trace eval có thể không kết luận được; điểm 0 là kết quả quan sát.

Campaign nhận `coverage_inventory` theo 7 chiều và `cases[].coverage_links` . Mỗi item có namespace, nguồn/version/digest. Mẫu số chưa khai báo trả null; số đã map, đã chạy, đúng kỳ vọng và có evidence được tách riêng.

`POST /v2/learning-datasets` tạo phiên bản train/validation từ job/check đã chạy; `GET /v2/learning-datasets` liệt kê bản của mình. Reviewer có phạm vi đọc `GET /v2/learning-datasets/{record_id}` và ghi `POST /v2/learning-datasets/{record_id}/reviews` với `dataset_digest`, `decision` và `note`. Duyệt độc lập, không ghi đè phiên bản, không tự kích hoạt Golden/Memory. Xem thao tác tại [Hoạt động TI](#) và [Duyệt dữ liệu learning](#).

POST `/v1/testing/changes`

v1 FROZEN · legacy — Deprecation/Link → /v2/artifact-jobs (không Sunset)

BODY · APPLICATION/JSON

Đường này nhận một trong hai body độc lập. Legacy change request dùng `changed` và `flow_id`:

+ Legacy change request (JSON)

Inline artifact submission dùng `schema_version: 1`; `source` định danh repository, số PR, `head_commit` cuối và `base_commit` tùy chọn. Mỗi artifact bắt buộc có `id`, dotted-numeric `version`, `content_type`, nội dung UTF-8 không rỗng, SHA-256 chữ thường khớp đúng bytes, và `criteria[]` gồm `{id, description}`.

`baseline` tùy chọn mang đúng `{version, content, sha256}`. Hoặc `canonical_link: {binding_id}` yêu cầu một binding do operator cấu hình sẵn; API này không cấp binding. Có thể gửi `idempotency_key` để retry cùng request; dùng lại key cho body khác trả 409. `policy` bắt buộc là `artifact-v1`. Body artifact không nhận `async`, `flow_id`, `environment` hay `role`.

Origin Browser / API-runner (HTTPS) là trường `environment_ref` trên hàng `CanonicalBindings` do operator sở hữu — server gán theo `binding_id`, không lấy từ body artifact. Binding không có trường này thì không bật explorer tools. Biến tiến trình `TI_ARTIFACT_ENVIRONMENT_REF` bị cấm: nó sẽ chuyển hướng mọi repository sang một origin.

Nhận 1–20 artifacts; nội dung `text/plain`, `text/markdown` hoặc `application/json`; tối đa 256 KiB UTF-8 mỗi artifact, 1 MiB tổng nội dung artifact/baseline, và 1–50 criteria mỗi artifact. `policy` bắt buộc là `artifact-v1` do server sở hữu.

+ Body artifact đầy đủ (JSON, bấm đã đúng)

Ví dụ chỉ chứng minh contract shape, không chứng minh một successful live Harness invocation. Phạm vi bằng chứng thực tế là nội dung contract đã nộp; hành vi runtime vẫn thiếu cho tới khi có quan sát được ghi nhận.

Artifact POST trên v1 luôn xếp hàng và trả 202 `{job_id, state, poll, preferred_poll}` kèm header Deprecation/Link (không Sunset/không ngày tắt). `poll` vẫn là `/v1/testing/changes/{job_id}` (hợp đồng v1); `preferred_poll` trở `/v2/artifact-jobs/{job_id}`. Poll v1 trả 202 khi `queued` / `running`; 200 `completed` với `{job_id, state, result, ...}` (allowlist v1); hoặc 503 `failed` với lỗi terminal đã làm sạch — kể

cả khi kho lưu `timed_out` / `cancelled` (chiếu thành `failed` trên wire v1). `result` giữ `source` , `policy_version` , kết quả từng artifact/check và `evidence/Harness provenance`; không phải bản ghi `legacy change_intelligence` .

ARTIFACT CANDIDATE · REVIEW VÀ MEMORY

CI vẫn chỉ submit artifact và đọc kết quả. Một key thường có canonical binding thuộc phạm vi của mình có thể nhận candidate máy sinh trong `result.artifacts[].canonical.candidates[]` (trạng thái `CANDIDATE`, chưa chạy, chưa duyệt) và tự ghi chúng thành bản ghi chờ duyệt bằng `POST /v1/testing/submissions/{job_id}/candidates` ; ghi tự động khi job hoàn tất hiện chỉ ở đường CI có receipt tin cậy. Cả hai thao tác không cần GitHub OIDC. Không gửi candidate hoặc role tự khai vào các lệnh review/thu hồi; không tự duyệt, đưa vào `GOLDEN` hay xuất bản `Memory`.

API	BODY	QUYỀN VÀ KẾT QUẢ
<code>POST /v1/testing/submissions/{job_id}/candidates</code>	không có body	<code>GET_RUN</code> và chủ job trong binding hiện hành; 201 danh sách candidate riêng tư
<code>GET /v1/testing/candidates?limit=50</code>	không có body; limit 1–100, mặc định 50	<code>READ_SUBMISSION_CANDIDATE</code> trong đúng phạm vi {owner, repository, binding_id} tươi của credential; 200 {candidates[], limit} chỉ metadata (record_id , repository , binding_id , job_id , artifact_id , candidate_id , candidate_status , review_decision , publication_state), xếp theo record_id, không cursor; không có phạm vi thì rỗng, không đếm bản ghi ngoài phạm vi
<code>GET /v1/testing/candidates/{record_id}</code>	không có body	<code>READ_SUBMISSION_CANDIDATE</code> và (chủ sở hữu còn <code>GET_RUN</code> , hoặc người duyệt hợp lệ trong phạm vi); 200 bản ghi

POST {decision, note} REVIEW_SUBMISSION_CANDIDATE và người duyệt hợp lệ trong phạm vi; 200 quyết định bất biến

/v1/testing/candidates/review/{record_id}/review

POST {note} REVIEW_SUBMISSION_CANDIDATE và người duyệt hợp lệ trong phạm vi; 200 sự kiện thu hồi

/v1/testing/candidates/review/{record_id}/revoke

Review/thu hồi giữ quyền owner hoặc authority trong danh sách reviewer của canonical binding, với credential delegated hoặc bootstrap. Người đã xác minh qua delegated cũng có thể dùng vai reviewer khi kho vai vừa nạp còn cấp đúng RoleGrant.review_scopes gồm {owner, repository, binding_id}; canonical binding đó phải còn tồn tại. Bootstrap chỉ dùng nhánh canonical; credential agent không được duyệt. Vai reviewer chỉ có hai capability candidate trên, không có quyền ADMIT_TEST_CASE để review legacy/gán nhãn, hay quyền đọc/chạy/liệt kê job. Kích hoạt vai mới cần nạp lại hoặc khởi động lại service phân giải danh tính; thu hồi scope có hiệu lực ngay ở lần kiểm quyền tiếp theo, kể cả publication, Memory reuse và hoàn tất job đang chạy. Không có quyền đọc phạm vi thì bản ghi có thật và không tồn tại đều bị che bằng 404. note bắt buộc, không rỗng, tối đa 1.024 ký tự; body không nhận trường lạ.

+ Body review (JSON)

+ Body revoke (JSON)

Candidate gốc vẫn mang trạng thái CANDIDATE ; quyết định nằm ở review.decision : GOLDEN, EXPERIMENTAL hoặc REJECTED. Review không được ghi đè; gửi lại đúng quyết định, người duyệt và note là retry. Không dùng endpoint legacy /v1/testing/candidates/review thay cho hợp đồng theo record_id này.

Khi AWS Memory được cấu hình, chỉ GOLDEN đã ghi thành công vào Memory và có manifest còn được phép sử dụng mới được chọn. Nếu publication lỗi, review đã lưu vẫn giữ

nguyên; retry review giống hết để hoàn tất publication. Kiến thức có nguồn, người duyệt, phạm vi và `knowledge_version = candidate_sha256` ; nội dung JSON gửi Memory tối đa 16.000 ký tự. Không tự đưa candidate chưa duyệt, chưa publish, khác phạm vi/version hay đã thu hồi vào đánh giá.

Lướt mới đọc đúng Memory record và namespace của manifest, đối chiếu hash rồi ghim nội dung; lỗi đọc hoặc không khớp thì không fallback sang SQLite. Bằng chứng truy xuất, context đã ghim và test thực sự chạy là ba bằng chứng riêng. Thu hồi ngăn sử dụng tiếp và chặn hoàn tất job đang dùng kiến thức đó; không sửa context hoặc kết quả lịch sử. Cùng người thu hồi gửi lại đúng note là retry; khác người hoặc khác note là xung đột, không phải huỷ thu hồi.

Luồng tái sử dụng kiến thức đã duyệt trong AWS Memory chưa hoàn tất nghiệm thu live. Bản ghi Memory tồn tại, job completed hoặc CI xanh riêng lẻ không chứng minh lướt sau đã sử dụng đúng kiến thức và không tự cấp quyền release.

LEGACY CHANGE REQUEST

TRƯỜNG	Kiểu		NGHĨA
change d	danh sách {kind, id, version} , 1 đến 16 phần tử	BẮT BUỘC	kind là artifact , scenario hay capability ; version là version MỚI. Version cũ tra từ register của service; trùng version đang có là 400 CHANGE_REJECTED (không có lineage)
flow_i d	chuỗi ≤ 256	BẮT BUỘC	khoá nối tới Jira, XBrain và Runtime; mọi bản ghi của lướt mang nó
gate	G2 · G4 · G5 · G6	tuỳ chọn, mặc định G5	cổng được hỏi; bảng lớp test bắt buộc theo cổng nằm trong engine
observa tions	{"T-...": {"result", "actual_output" }}	TUỖY CHỌN	quan sát bạn tự đo cho một test; result là pass , fail , abstain hay not_evaluable . Có quan sát thì dùng quan sát, không thì hỏi agent
environ ment	đối tượng {ref}	TUỖY CHỌN	môi trường thật của bản PR (URL http/https): evidence ghi environment = ref ; adapter API runner (bật bằng

TI_ADAPTER_API_RUNNER=on) gọi vào đó qua Gateway và trích trace làm bằng chứng

async

true · false

tuỳ chọn, mặc định false

true ⇒ trả 202 {job_id, poll} ngay, rồi GET /v1/testing/changes/{job_id} trả 202 {state: running} tới khi có 200 . Dùng qua địa chỉ công khai: agent đo và sinh test mất vài phút, CloudFront cắt ở 30 giây

PHẢN HỒI

200

Đã trả lời · chín câu trong summary , cộng candidate, rủi ro, coverage, quality, gap

-

TRƯỜNG	Kiểu	NGHĨA
run_id	chuỗi	mã bản ghi change_intelligence , dùng cho GET bên dưới
summary.what_changed	danh sách	artifact:RCA-CPU-CONTENTION@1.0→1.1
summary.what_is_impacted	đối tượng	direct, indirect, releases, customers
summary.what_tests_are_required	đối tượng	kế hoạch, tier rủi ro, test bắt buộc, lớp còn trống
summary.what_tests_run	danh sách	test, adapter (agentcore , observed , none), kết cục
summary.what_gate_is_blocked	đối tượng	recommendation PASS · PASS_WITH_CONDITIONS · DO_NOT_PASS, reason , confidence , blocked_by
summary.who_has_authority	chuỗi	EVALUATION_AUTHORITY cho G5: TI khuyến nghị, không quyết
candidates	danh sách	test máy sinh cho lớp còn trống, trạng thái CANDIDATE ; duyệt ở đường dưới

```
{
  "run_id": "06f87bf1-...",
```

```

"summary": {
  "what_changed": ["artifact:RCA-CPU-CONTENTION@1.0→1.1"],
  "what_is_impacted": {"direct": ["artifact:RCA-CPU-CONTENTION"], "indirect":
["capability:Incident Investigation", "scenario:SCN-CPU-SATURATION"], "releases":
["REC-II-0.8"], "customers": "unknown: no deployment registry"},
  "what_tests_are_required": {"plan": "TPL-eb269a40", "risk_tier": "CRITICAL",
"mandatory": ["T-CONTRA-CPU-003", "..."], "uncovered_classes": ["Negative", "Partial",
"Abstention"]},
  "what_tests_ran": [{"test": "T-CONTRA-CPU-003", "adapter": "agentcore", "result":
"pass"}, "..."],
  "what_failed": [],
  "what_is_not_evaluable": ["T-REG-CPU-010"],
  "what_evidence_exists": ["TE-..."],
  "what_gate_is_blocked": {"gate": "G5", "recommendation": "DO_NOT_PASS", "reason":
["mandatory test T-REG-CPU-010 not evaluable: ...", "Negative evidence coverage below
threshold: 0 planned, 0 passed, 0 not evaluable"], "confidence": 0.86, "blocked_by":
["T-REG-CPU-010"]},
  "who_has_authority": "EVALUATION_AUTHORITY",
  "what_should_happen_next": ["produce measurable evidence for mandatory test T-REG-
CPU-010 (Regression) – owner: Testing Intelligence Lead ...", "review candidate TC-CAND-
1de3d65b (Negative) → GOLDEN / EXPERIMENTAL / REJECTED ..."]
},
  "candidates": [{"candidate_id": "TC-CAND-1de3d65b", "status": "CANDIDATE",
"test_case": {"...": "..."}}]
}

```

400

CHANGE_REJECTED · version trùng version đang có, hoặc candidate sai hình dạng

+

403

AUTHORITY_INSUFFICIENT · vai không mang run_suite (reader)

+

503

AGENTCORE_UNAVAILABLE ·
EVALUATOR_UNAVAILABLE· agent chưa cấu hình, hoặc ổ dữ liệu của
service chưa có register

+

GET

/v1/testing/changes/{job_id}

v1 FROZEN · poll artifact job (FAILED = 503); mã run legacy vẫn đọc lại qua cùng path

Artifact job (v1): mã 32 ký tự hex trả projection allowlist {job_id, state, created_at, updated_at, source, artifacts, result, error}. Terminal failed (kể cả chiếu từ

timed_out/cancelled) ⇒ **HTTP 503**. Không có `trace_id` / `result_status` / `lifecycle` trên v1 — đọc các trường đó qua [GET /v2/artifact-jobs/{job_id}](#). (200 + state). Khi completed, `result.source` khớp PR/head đã nộp. Projection không trả credential hay nội dung bí mật.

Legacy run: mã run legacy trả bản ghi `change_intelligence` bên dưới. Hai response này là hai contract độc lập.

ARTIFACT JOB COMPLETED · VÍ DỤ TERMINAL KHÔNG CÓ EVALUATOR

+ Ví dụ job completed · JSON đầy đủ

Ví dụ trung lập này kết thúc `completed` nhưng `recommendation` vẫn `DO_NOT_PASS` : chỉ integrity deterministic đạt, các check cần evaluator là `not_evaluable` . Completed nghĩa là job đã kết thúc, không có nghĩa artifact được duyệt.

200

Bản ghi · vô (danh tính, vai lúc gọi, thời điểm, flow_id) và thân so sánh được

-

TRƯỜNG	KIỂU	NGHĨA
kind	chuỗi	change_intelligence
arguments.test_run_ids	danh sách	bản ghi test_run của từng test, cùng flow_id
payload	đối tượng	chín câu và phần phụ, không có thời điểm và id lượt chạy: hai lượt giống nhau so bằng nhau

```
{
  "run_id": "06f87bf1-...",
  "kind": "change_intelligence",
  "identity_subject": "entra:...",
  "role_at_invocation": "evaluator",
  "created_at": "2026-09-04T10:16:54Z",
  "flow_id": "FLOW-REF-X00R-002",
  "arguments": {
    "changed": ["..."],
    "gate": "G5",
    "test_plan_id": "TPL-eb269a40",
    "test_run_ids": ["..."]
  },
  "payload": {
    "...": "..."
  }
}
```

404

CHANGE_NOT_FOUND · không có bản ghi, hoặc run_id thuộc bản ghi loại khác

+

POST /v1/testing/candidates/review

Duyệt một candidate: GOLDEN, EXPERIMENTAL hay REJECTED. Máy sinh, người nhận

BODY · APPLICATION/JSON

TRƯỜNG	KIỂU		NGHĨA
candidate	đối tượng	BẮT BUỘC	chép nguyên một phần tử candidates[i] của POST ở trên
decision	GOLDEN · EXPERIMENTAL · REJECTED	BẮT BUỘC	quyết định của người duyệt
note	chuỗi ≤ 1024	BẮT BUỘC	vì sao
flow_id	chuỗi ≤ 256	BẮT BUỘC	khoá nối của bản ghi candidate_review

PHẢN HỒI

200

Đã duyệt · bản ghi candidate_review là bằng chứng

-

TRƯỜNG	KIỂU	NGHĨA
status	chuỗi	quyết định đã ghi
reviewed_by	chuỗi	danh tính người duyệt, lấy từ key, không từ body

```
{ "run_id": "...", "candidate_id": "TC-CAND-1de3d65b", "status": "EXPERIMENTAL", "reviewed_by": "entra:...", "review_note": "cần dataset", "test_case": { "...": "..." } }
```

403

AUTHORITY_INSUFFICIENT

· vai không mang admit_test_case (chỉ owner, authority); danh tính agent bị từ chối kể cả khi vai cho phép

+

POST /v1/testing/labels

Gán nhãn một quan sát: correct, wrong hay missing_evidence. Bản ghi bất biến; nhãn là ngữ cảnh cho đánh giá tiếp theo

BODY · APPLICATION/JSON

TRƯỜNG	Kiểu		NGHĨA
test_run_id	chuỗi ≤ 256	BẮT BUỘC	một phần tử test_run_ids của POST /v1/testing/changes, hoặc cột <i>Bản ghi</i> trên trang Hoạt động TI
verdict	correct · wrong · missing_evidence	BẮT BUỘC	quan sát đúng, sai, hay không đủ bằng chứng để nói
corrected_result	pass · fail · abstain · not_evaluable	với wrong	kết cục đúng — bắt buộc khi wrong, cấm với hai nhãn kia
note	chuỗi ≤ 1024	BẮT BUỘC	vì sao

PHẢN HỒI

201

Đã ghi nhãn · bản ghi observation_label giữ nhận xét và corrected_result làm ngữ cảnh; lượt tiếp theo vẫn đo mới qua adapter. Nhãn không thay kết quả đo mới, không ghi đè – nhãn bằng chứng cũ và không biến thành phép đo adapter: label:<label_id>

TRƯỜNG	KIỂU	NGHĨA
label_id	chuỗi	id nhãn — cũng là run_id của bản ghi
labelled_by	chuỗi	danh tính người gán, lấy từ key, không từ body

```
{"run_id": "...", "label_id": "...", "test_run_id": "...", "test_case_id": "T-SAFE-CPU-004", "target": "artifact:RCA-CPU-CONTENTION@1.1", "observed_result": "pass", "verdict": "wrong", "corrected_result": "fail", "note": "...", "labelled_by": "entra:..."}

```

403

AUTHORITY_INSUFFICIENT

· vai không mang admit_test_case (chỉ owner, authority);

danh tính agent không gán được nhãn cho quan sát của chính nó

+

404

·

409

·

400

CHANGE_NOT_FOUND

· 404: không có bản ghi mang test_run_id ; 409: đúng nhãn này đã ghi (bản ghi bất biến — muốn nói khác thì ghi nhãn mới

· CHANGE_REJECTED với lý do mới); 400: bản ghi không phải test_run , wrong không kèm sửa, hay sửa thành chính kết cục đã quan sát

+

GET

/v1/testing/labels/dataset

Mọi nhãn dưới dạng dataset AGENTCORE_EVALUATION_PREDEFINED_V1 cho evaluator TIRunnerGroundedness — tính từ kho bằng chứng lúc đọc

PHẢN HỒI

200

Dataset

· một example cho mỗi nhãn: scenario_id = label id, turns[0].input = tài liệu agent đã thấy, turns[0].expected_response = kết cục người xác nhận, assertions = nhãn và lý do

-

TRƯỜNG	KIỂU	NGHĨA
schema_type	chuỗi	AGENTCORE_EVALUATION_PREDEFINED_V1

examples mảng các example, thứ tự ghi

```
{ "schema_type": "AGENTCORE_EVALUATION_PREDEFINED_V1", "evaluator": "TIRunnerGroundedness", "examples": [ { "scenario_id": "...", "turns": [ { "input": "{...}", "expected_response": "result=fail" } ], "assertions": [ "[wrong] ..." ] } ] }
```

GET

/v1/testing/attachments/{sha256}

Ảnh bằng chứng (TI-06) theo hash: tra bản ghi test_run mang attachment này, đọc kho tệp, chỉ trả khi SHA-256 của byte khớp bản ghi

PHẢN HỒI

200	Byte ảnh	· content-type lấy từ bản ghi (thường image/png); header x-attachment-key và x-attachment-sha256	+
404	ATTACHMENT_NOT_FOUND	· 404: không bản ghi nào mang hash này; 503: bản triển khai không có kho tệp, kho không trả được khoá, hoặc byte đọc về KHÔNG khớp hash (ảnh bị thay hay hỏng	+
503	ATTACHMENT_UNAVAILABLE	— không bao giờ trả một ảnh sai hash)	

05 · ĐỌC KẾT QUẢ

Bốn lớp trong một kết quả. Không gộp làm một.

Một job completed chứa bốn thứ khác nhau, ở bốn trường khác nhau. Đừng để "completed", "mọi check pass" và "CI xanh" trộn thành "release PASS".

LỚP	TRƯỜNG TRONG PHẦN HỒI	NGHĨA	KHÔNG CÓ NGHĨA
1 · Trạng thái job	state và mã HTTP của poll	202 queued/running: chưa xong. 200 completed: đã kết thúc, có result . 503 failed: lỗi terminal có tên trong error .	completed không nói artifact đạt hay không
2 · Check được điều phối · đã đo thật	result.artifacts[].checks[]	Mỗi phần tử là một check được điều phối: result (pass · fail · not_evaluable), adapter (deterministic , none , hoặc tên runtime Harness), evidence_scope , evidence_id , evidence[] , invocations[] . Chỉ hàng có adapter khác none và kết cục pass/fail là test đã đo thật; đừng đếm mọi hàng trả về là test đã chạy.	not_evaluable không phải pass; adapter: none nghĩa là không ai đo
3 · Candidate máy sinh	result.artifacts[].canonical_candidates[] và canonical.not_run[]	Chỉ có khi artifact gắn canonical binding hợp lệ. Mỗi candidate là một đề xuất test cho lớp còn trống, trạng thái CANDIDATE , chưa chạy, chưa duyệt. not_run[] là test có trong bundle nhưng lượt này không chạy, kèm lý do.	candidate không phải evidence, không tính vào recommendation, không tự vào bộ đề
4 · Khuyến nghị	result.artifacts[].status , recommendation , recommendation_reasons[] , blocked_by[] , gaps[]	Chính sách artifact-v1 của server tính từ lớp 2: PASS · PASS_WITH_CONDITIONS · DO_NOT_PASS, kèm từng lý do và check nào chặn.	không phải quyết định release; người có thẩm quyền quyết

TRÍCH ĐOẠN CANONICAL KHI ARTIFACT CÓ BINDING · CANDIDATE CÒN CHỜ, CHƯA CHẠY

+

Trích đoạn canonical · candidate còn chờ (JSON)

Trong ví dụ completed ở mục [Artifact của PR](#), canonical là null nên không có lớp 3. Khi có, trạng thái của candidate giữ CANDIDATE cho tới khi một người duyệt ghi quyết định qua [review](#)

[theo record_id](#); TI không tự thực thi và không tự duyệt candidate. Trang [Hoạt động TI](#) hiển thị các lớp này tách nhau cho operator; người tích hợp đọc job của mình qua API có scope.

06 · KHI BỊ TỪ CHỐI

Mã thật của hai endpoint, và ai gỡ được.

Mỗi mã HTTP đi kèm một `reason_code` đã khai và một `remediable_by`; thân từ chối chỉ nêu tên trường, không nêu giá trị. Bảng dưới là hai endpoint của đường artifact; hai mặt legacy có bảng riêng ở mục Trạng thái.

KHI BỊ TỪ CHỐI · MÃ THẬT CỦA HAI ENDPOINT TRÊN

HTTP · REASON_CODE	KHI NÀO	AI GỠ · LÀM GÌ
422 AUTHORITY_INSUFFICIENT	thiếu header, key sai, hết hạn hoặc đã thu hồi; thân còn có <code>result: INSUFFICIENT_EVIDENCE</code> . Đây là lời từ chối danh tính của API (không phải 401)	CALLER · lấy key mới ở mục Lấy key rồi gọi lại
403 AUTHORITY_INSUFFICIENT	danh tính hợp lệ nhưng vai không mang quyền, hoặc credential CI gắn repository khác với <code>source.repository</code>	PLATFORM · xin đúng vai; CI gửi đúng repository của credential
400 UNDECLARED_FIELD_REJECTED	body sai schema: bấm không khớp nội dung, version không phải số có dấu chấm, thiếu hay thừa trường, <code>content_type</code> lạ, vượt trần kích thước, JSON hỏng với <code>application/json</code> . Thân chỉ nêu <code>rejected_fields[]</code> , không nêu giá trị	CALLER · tính lại bấm ở bước 2, đối chiếu schema ở mục Artifact của PR
422 CHANGE_REJECTED	body đúng schema nhưng service từ chối, ví dụ <code>canonical_link.binding_id</code> không thuộc phạm vi hay nội dung mang thứ có dạng credential; thân chỉ nói artifact submission bị từ chối	CALLER · bỏ binding không thuộc mình, làm sạch nội dung
409 CHANGE_REJECTED	<code>idempotency_key</code> đã dùng cho một body khác	CALLER · key mới cho body mới; retry cùng body thì giữ key và nhận lại cùng job

NHÓM	TRÁCH NHIỆM	CHẠY Ở ĐÂU
S01 · Target Registry	đối tượng nào, version nào (kind:id@version); ghim bối cảnh trước khi nhận	ghim ở edge · phân giải ở Runtime
S02 · Change Detector	cái gì đổi, từ version nào sang version nào	Runtime
S03 · Impact Engine	direct · indirect · runtime · regression · release · customer	Runtime
S04 · Risk Engine	RegressionRisk năm thừa số; tier quyết độ sâu	Runtime
S05/S06 · Planning + Generation	test nào bắt buộc, lớp nào còn trống, candidate cho lớp trống — người duyệt, máy không tự nhận	Runtime
S07 · Orchestration	nhận artifact, xếp hàng bền, điều phối check qua adapter và Harness	nhận vào + queue ở edge · orchestrate ở Runtime
S08 · Evidence Store	bằng chứng bất biến, ghi một lần	ghi bền ở edge · dựng bằng chứng ở Runtime
S09 · Gate Recommendation	khuyến nghị của máy — không phải Gate Decision của Authority	Runtime
S10 · Production Learning	EvaluationGap → ImprovementCandidate, qua review của người	phân tích ở Runtime · xác thực, ghi bền và review ở API edge

VAI AGENTCORE · MỨC ĐÃ ĐẠT

Bốn mức tách nhau và không suy ra được lẫn nhau: **mã đã đấu** (có trong kho) → **đã triển khai** (đang chạy) → **pilot đã đạt** (một lượt gọi thật đã đạt, ở phạm vi pilot) → **đã nghiệm thu** (một lượt nghiệp vụ TI được chấp nhận). Tạo được tài nguyên hay chạy được pilot **không** chứng minh mức nghiệm thu.

VAI	MỨC ĐÃ ĐẠT	BẰNG CHỨNG VÀ GIỚI HẠN
Runtime	ĐÃ TRIỂN	API chuyển pipeline artifact sang TIJobRunner ; shadow đã hoàn tất qua bộ kiểm wire. Hoàn tất tính toán không đồng nghĩa case đạt hoặc job API mới

	KHAI	đã có bằng chứng thực thi. Phân tích production của S10 gọi cùng TJobRunner; API giữ xác thực, ghi bền và review.
Gateway	ĐÃ TRIỂN KHAI	Hai target: Lambda HTTP và Runtime MCP Playwright. Lượt gọi qua Gateway trong đường artifact chưa nghiệm thu.
Memory	ĐÃ TRIỂN KHAI	Harness Memory ACTIVE; publish và ghim đi qua quản trị của API. Tái sử dụng kiến thức chưa hoàn tất nghiệm thu.
Identity	ĐÃ TRIỂN KHAI	Bốn workload identity đã tồn tại. Vai của caller vẫn tra ở kho phía server, không nhận từ trường do caller tự khai.
Observability	ĐÃ TRIỂN KHAI	Harness tự phát OTel; provenance đã đầu dây — job, session, request, layer và digest của tool đều được ghi. Trace đầu-cuối của một job TI và nghiệm thu eval vẫn đang chờ.
Evaluations	PILOT ĐÃ ĐẠT	Một phiên nativeCodeHarness thật đo GoalSuccessRate 1.0 — kết quả của evaluator, không phải nghiệm thu một job TI.
Code Interpreter · native	PILOT ĐÃ ĐẠT	Pilot thật đã đạt. Mô tả tool do server sở hữu và cố định, bật bằng biến môi trường; không bao giờ dựng từ nội dung artifact.
Browser · native	PILOT ĐÃ ĐẠT	Browser tùy biến đã nối với Playwright CDP trong Runtime MCP qua Gateway. Năm tool đã chạy trên origin công khai trung tính, phiên đã đóng; ảnh PNG trên S3 đã được đọc lại và kiểm hash, kích thước, MIME. Đó là pilot dịch vụ, chưa phải bằng chứng gắn job/case TI. Origin ảnh job/case phải là CanonicalBindings.environment_ref đã phạm vi — không phải biến môi trường tiến trình toàn cục.
Policy	ĐÃ TRIỂN KHAI	Policy engine và permit policy ACTIVE ở chế độ LOG_ONLY (fail-on-any-findings) trên sáu hành động Gateway; Gateway và cả hai target READY. Đây là đầu dây quan sát , không phải cưỡng chế từ chối — không có tuyên bố ENFORCE.

Chưa vai nào ở mức **đã nghiệm thu** cho đường artifact TI. Model suy luận là Bedrock Claude do Harness gọi; Claude/Cursor trên máy phát triển là công cụ phát triển, không phải bộ chạy production. Trạng thái Policy, Memory và Identity ở đây là mức đo được tại thời điểm dựng trang, không phải cam kết vận hành.

BẢN ĐỒ ĐIỂM GẮN · MỖI NHÓM TI DÙNG DỊCH VỤ AGENTCORE NÀO

Một dòng cho một nhóm. Dịch vụ gộp trong ô, vì phần lớn nhóm dùng nhiều hơn một: Runtime quản lý chạy pipeline artifact và phép tính production của S10; API giữ xác thực, ghi bền và review. Bảng chỉ rõ phần đang chạy và điểm dịch vụ có thể gắn thêm. Cột cuối ghi điều kiện phải có trước và trạng thái hiện tại bằng lời thường — đọc thẳng, không quy ra một nhãn mức nào.

NHÓM	DỊCH VỤ AGENTCORE GẮN VÀO	VÀO	RA	TIÊN QUYẾT VÀ TRẠNG THÁI
S01 · Target Registry	Runtime (phân giải và xác minh bản ghim) · Memory (quản trị tại API edge) · Registry — đề xuất	kind:id@version đã ghim; record_id và namespace của bản ghi kiến thức đã duyệt	target đã phân giải; digest và liên kết bản ghim đã kiểm; hiệu lực Memory do API kiểm	PinnedContext tự ki digest khi phân tích; quyền nhận vào và > thực ở lại API edge. Memory mới có mã, và tài nguyên đã cấu — chưa có lượt pub → tái dùng → thu hể Registry là catalog tool/skill của agent, KHÔNG phải kho đố tượng kiểm của TI; k có mã trong bản dựr này.
S02 · Change Detector	Runtime (declare)	Register đã ghim và danh sách VersionRef mới; version cũ tra trong Register	tập thay đổi: cái gì đổi, từ đâu sang đâu	Register và ref mới l lệ; đối tượng chưa c trong Register được ADDED, không đòi ị có version cũ
S03 · Impact Engine	Runtime (build_impac t_set)	tập thay đổi từ S02 và đồ thị phụ thuộc đã ghim	ImpactSet sáu trực: direct · indirect · runtime · regression · release · customer	S02 xong. Đây là ph tính trên đồ thị, khô tool thực thi nào: Br và Code Interpreter ở S07, không phải ở

S04 · Risk Engine	Runtime (<code>assess</code>)	Register, TestCase, ChangeSet và ImpactSet (<code>evaluate.py:228</code> truyền <code>impact_set</code>); deployments, history, usage nếu có	RiskAssessment cho từng thay đổi: điểm năm thừa số và tier quyết độ sâu chạy	Register và ChangeSet hợp lệ; assess tự tin động cho từng thay đổi, không nhận ImpactSet của S03 làm đối số
S05/S06 · Planning + Generation	Runtime (<code>canonical_plan</code>) · Harness (<code>propose_test_case</code>) · Bedrock Claude	Planning: ChangeSet, ImpactSet, tier, TestCase và yêu cầu bằng chứng; generation: TestPlan, VersionRef đích và nội dung artifact đã ghim	TestPlan gồm test bắt buộc và lớp còn trống; Harness đề xuất CandidateTestCase cho lớp trống, luôn CANDIDATE và chưa chạy	Luồng canonical trong TIJobRunner dùng HarnessRunner.generate_candidate và tool <code>propose_test_case</code> xuất candidate từ artifact đã resolve — đầu để <code>transports/agentcore_missions.py:350</code> , Bỏ theo mặc định; đặt <code>TI_AGENTCORE_AGENT_CORE_ATE=off</code> để quay về template, và adapter không phải Harness dùng template. S05 lập kế hoạch theo cách của TI. Trường <code>generator</code> phân biệt candidate do AgentCore sinh với template dự phòng; template KHU phải bằng chứng mới đã sinh. Candidate phải qua người duyệt, mà không tự nhận. Schedule tool do server sở hữu Bedrock Claude là n suy luận, còn Claude Cursor trên máy phả là công cụ lập trình v

S07 · Orchestration	Runtime (orchestrat e + HarnessRunne r) · Gateway (hai target) · Identity · Policy · Code Interpreter và Browser — thực thi tool nằm ở đây	TestPlan, TestCase, VersionRef đích và adapter theo lớp; job đã ghim và tool pattern do server cho phép	TestRun và TestEvidence của từng case; adapter có thể trả trace HTTP, kết quả chạy mã hoặc ảnh PNG có sha256 — không mặc định mọi case có ảnh	Đúng một lượt InvokeAgentRuntime API edge; cấu hình c môi trường của tiến không nhận từ bên c (ARN/qualifier/name Runtime — không pl origin Browser). Orig explorer là binding.environment ; không đặt TI_ARTIFACT_ENVIROI REF . Shadow đã trả quả hợp lệ qua bộ k wire; hoàn tất tính to không đồng nghĩa h thống bị kiểm đạt, và chưa chứng minh m API mới được lưu c bằng chứng thực thi Identity: bốn workloa identity tồn tại, chưa vòng đời credential c đủ. Policy chạy LOG trên sáu hành động Gateway — ghi lại, c chặn. Code Interpre lượt gọi tool quan sá được trong shadow chưa phải một case candidate đạt. Brow cả năm tool Browse quản lý qua Gatewa đã chạy được, mỗi l một phiên riêng và p nào cũng đóng lại; ả chụp lưu trên S3 đã hash, kích thước, ki MIME và đúng là PN Đó là chứng minh đi hướng và ảnh PNG
------------------------	--	--	--	---

một origin công khai
tính — **không phải** l
chứng đã ghép vào
job hay một case củ

S08 · Evidence Store	Runtime (dựng bằng chứng) · Observability	quan sát của từng case; job · session · request · digest của layer và tool	tham chiếu tệp bằng chứng cho kết quả job; bằng chứng do tool sinh (ảnh chụp) nằm trên S3; trace OTel tương quan	Thẩm quyền lưu job kết quả nằm ở API e ghi bèn, bất biến, m Điều đó không có n Runtime không ghi c tool quản lý tự sinh lưu bằng chứng củ lên S3 (ảnh chụp); h việc khác nhau: lưu job/kết quả có thẳm quyền ở API, sinh về bằng chứng ở Runtime/tool. Observability có sẵn job/session/request/ layer và telemetry q còn thiếu một trace t vện của một job TI v bằng chứng eval.
S09 · Gate Recommendation	Runtime (recommend) · Evaluations	Gate, TestPlan, TestRun, TestEvidence, TestCase và phạm vi bằng chứng bắt buộc; trace phiên nếu nối Evaluations	GateRecommendation của máy; điểm Evaluations là tín hiệu chất lượng agent riêng, không nằm trong kết quả recommend hiện tại	recommend tính từ r evidence trong bộ n trước khi API ghi bèn quả; API vẫn phải xé minh và lưu bằng ch trước khi công bố k có thẩm quyền. Gate Decision thuộc Auth là người. Evaluation tín hiệu chất lượng a dựng trên trace và session — pilot trung đã chạy ở phạm vi session, chưa tự ghi vào recommend; nó không chạy case, kh quyết pass/fail của h

thống bị kiểm và khẩ
thẩm quyền công.

S10 · Production Learning	Runtime (detect_gaps + propose) · API governance (xác thực, ghi bên, review và quản trị Memory) · Evaluations (điểm nổi phản hồi) · Optimization — đề xuất	API analyze_production đọc record_id đã xác thực; gửi ProductionEvidence và TestCase đã lưu cùng owner, repository, binding_id, record_id, hai digest và deadline sang cùng TJobRunner	Runtime detect_gaps → EvaluationGap, propose → ImprovementCandidate; API đối chiếu các binding và digest, kiểm tra bản ghi còn hiện hành rồi ghi kết quả	Phép tính production qua nhánh analyze_production cùng TJobRunner, k gọi model. API không giao dịch ghi trong l Runtime; lỗi hoặc bị thay đổi thì không g phân tích, không âm tính lại ở API. Publis thu hồi Memory cũn API quản trị; qua rev của người, không tự thành GOLDEN, khẩ cấp quyền phát hàn Memory ở đây cũng là mã, test và tài ngu — chưa có lượt pub → tái dùng → thu hể Optimization tác độnn cấu hình agent (syst prompt, mô tả tool) c khuyến nghị từ trace xem lại, gói cấu hình version và thử A/B; k bao giờ tự ký case v đề GOLDEN. Không mã Optimization trorbản dựng này.
---------------------------	---	--	--	--

Thực thi tool kiểm hệ thống — Code Interpreter và Browser — thuộc S07. S03 chỉ tính trên đồ thị phụ thuộc, không thực thi Browser/Code; các nhóm khác vẫn có thể gọi dịch vụ như Memory, Harness hoặc Evaluations theo vai trò. Định nghĩa từng dịch vụ AgentCore đọc ở [tài liệu tổng quan Amazon Bedrock AgentCore](#).

TESTING INTELLIGENCE · EVALUATION SERVICE

TRANG CHỦ · API · SƠ ĐỒ · HOẠT ĐỘNG TI · FAQ · ĐĂNG NHẬP MICROSOFT ĐỂ TẠO KEY